

EduGenie: Google Gemini Powered Learning Assistant

Project Description:

EduGenie is an AI-powered learning assistant designed to help learners understand academic topics through conversational interaction. It uses Google Gemini to generate explanations, answer learning questions, summarize concepts, and support study-oriented queries in a simple and accessible interface.

The system is intended to provide fast, personalized learning assistance. A learner enters a question or topic, EduGenie sends the request to the Gemini-powered service, and the response is presented in the learning interface. The project focuses on clear explanations, easy interaction, and useful study support.

Team Details

Date	02 October 2026
Team ID	SWTID-2026-7991
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Team Leader	Mohamed Thariq
Members	Nandha Kumar, Muniyappan, Mohanakrishnan

Scenarios

Scenario 1: A student enters a difficult academic topic and requests a simple explanation. EduGenie returns a structured explanation suitable for learning.

Scenario 2: A learner asks for a short summary of a topic before revision. EduGenie generates a concise study-oriented response.

Scenario 3: A student asks follow-up questions about the same concept. EduGenie supports conversational learning by responding to the new query.

Technical Architecture

Description:

EduGenie follows a modular application architecture consisting of a user interface, application/backend layer, and Google Gemini AI integration. The frontend collects learner questions and displays responses. The backend manages request handling, validation, and communication with the AI service. Gemini provides the generative learning response.

Layer	EduGenie Component	Responsibility
Presentation	Web learning interface	Accept questions and display AI-generated answers
Application	Backend / API layer	Validate input, manage requests and responses
AI Integration	Google Gemini API	Generate explanations and learning responses
Configuration	Environment variables	Store API configuration securely

Note: If a database is added to the final implementation, an ER diagram and database description should be included with the submission.

Project Workflow

Milestone 1: AI Model and Architecture

Identify the learning requirements, select Google Gemini as the AI service, and define how the user interface, backend, and Gemini API communicate.

Milestone 2: Core Learning Features

Develop question answering, concept explanation, summarization, and conversational learning interactions.

Milestone 3: Backend Development

Implement request handling, input validation, Gemini API communication, response processing, and error handling.

Milestone 4: Frontend Development

Create a responsive learning interface with chat interaction, study navigation, and readable AI responses.

Milestone 5: Testing and Deployment

Run functional and performance checks, verify AI responses, test concurrent requests, and prepare the application for deployment.

Milestone 6: Conclusion

Review the completed learning assistant, document observations, and identify future improvements.

Milestone 1: AI Model Selection and Architecture

This milestone focuses on defining the educational purpose of EduGenie and selecting Google Gemini as the generative AI service. The architecture is planned around a clear flow from learner input to backend processing and AI-generated response.

Activity 1.1: Configure Google Gemini API

Create and configure the required Gemini API credentials. Store the key through environment configuration rather than hard-coding sensitive credentials in source files.

Activity 1.2: Define Learning Use Cases

Identify the main learning tasks: answering academic questions, explaining concepts in simple language, summarizing topics, and supporting follow-up questions.

Activity 1.3: Define Application Architecture

Document the interaction between the frontend, backend/API layer, and Google Gemini service.

Activity 1.4: Development Environment

Prepare the project environment, dependencies, configuration files, and local run setup.

Activity 1.1: Google Gemini API Configuration

EduGenie requires an API credential for communication with the Google Gemini service. The credential should be kept outside the source code and loaded through an environment variable.

Recommended configuration approach

1. Create or obtain the required Gemini API credential from the appropriate Google AI developer service.
2. Create a local environment configuration file such as **.env**.
3. Store the credential using an environment variable such as **GEMINI_API_KEY**.
4. Load the variable at application startup.
5. Add the environment file to **.gitignore** so that the secret is not committed to a public repository.

Security note: Never place a real API key in this documentation or screenshots.

Activity 1.2: Learning Model and Prompt Design

The learning assistant should use prompts that clearly describe the learner's request and the desired response style.

Prompt design considerations

- Identify the learner's question or topic.
- Request simple, accurate and structured explanations when appropriate.
- Ask the model to use examples or step-by-step explanations for difficult concepts.
- Keep responses aligned with the user's requested level or format.
- Handle follow-up questions using the current conversation context where supported.

Expected response structure

Input	Processing	Output
Question / topic	Validate and construct Gemini request	Learning explanation
Request for summary	Create concise prompt	Short study summary
Follow-up question	Use conversational context	Context-aware response

Activity 1.3: Application Architecture Details

Frontend responsibilities

Provide the learning chat interface, input controls, navigation and response display.

Backend responsibilities

Receive learner requests, validate input, call the Gemini service, handle errors, and return the generated response.

AI integration

The backend communicates with Google Gemini using the configured API credentials and sends a structured prompt containing the learner's request.

Data flow

Learner → EduGenie UI → Backend/API → Google Gemini → Backend/API → EduGenie UI → Learner

Activity 1.4: Development Environment

Prepare the development environment before implementing the application.

- Install the required programming language/runtime and package manager.
- Create the project directory and virtual environment where applicable.
- Install the project's dependency packages.
- Configure the Gemini API key through environment variables.
- Organize frontend, backend, configuration, and documentation files.
- Verify that the application starts successfully before beginning integration testing.

Milestone 2: Core Learning Functionalities

Milestone 2 focuses on implementing the main learning-assistant functions.

Feature	Description	Expected Result
AI Question Answering	Answer learner questions using Gemini	Relevant learning response
Concept Explanation	Explain difficult topics in simpler language	Easy-to-understand explanation
Summarization	Condense a supplied topic or content	Short revision-oriented summary
Follow-up Learning	Respond to related follow-up questions	Continuous conversational support

Input validation should be applied before sending requests to the AI service. Empty or invalid requests should return a clear message instead of an unnecessary API call.

Milestone 3: Backend Development

The backend serves as the control layer between the EduGenie interface and Google Gemini.

Core responsibilities

1. Receive the learner's request.
2. Validate the request and required fields.
3. Build the prompt sent to Gemini.
4. Call the Gemini API.
5. Process the returned response.
6. Return the result to the frontend.
7. Handle API failures, invalid input and service errors safely.

Testing focus: Verify both successful and error paths.

Backend Request and Response Handling

Typical request flow

Step	Action
1	Frontend submits learner question
2	Backend validates input
3	Backend prepares the Gemini prompt
4	Gemini generates the learning response
5	Backend receives and processes the response
6	Frontend displays the result

Error handling

The application should provide a user-friendly error message if the request is empty, the API configuration is unavailable, the AI service returns an error, or the response cannot be processed.

AI Response Integration

EduGenie should integrate the Gemini response into the learning interface without exposing secret configuration values.

Response quality checks

- Confirm that a response is received for a valid question.
- Verify that the answer is displayed in the correct chat area.
- Verify that long responses remain readable.
- Test follow-up questions and context handling.
- Test error messages when the AI service is unavailable.

Expected result: Learners should receive a clear response or a meaningful error message for every submitted request.

Milestone 4: Frontend Development

The frontend is designed as a learning-oriented conversational interface. The goal is to make asking questions and reading answers simple.

Activity 4.1: User Interface

Provide a home area, chat interaction, study-oriented navigation, input field, send action, and readable response cards.

Activity 4.2: Dynamic Rendering

Render new learner questions and AI responses dynamically so the user can continue the conversation without manually refreshing the page.

Usability considerations

Use clear labels, readable typography, sufficient spacing, responsive layout, and visible loading/error states.

Frontend Interaction and Response Display

The learning interface should support a simple interaction cycle.

UI Element	Purpose
Question Input	Enter the learner's question or topic
Send Button	Submit the question to EduGenie
Loading Indicator	Show that the request is being processed
User Message	Display the learner's submitted question
AI Response	Display the Gemini-generated learning answer
Navigation	Provide access to learning-related sections

The supplied EduGenie project screenshot can be used as visual evidence of the interface and testing presentation.

Milestone 5: Testing and Deployment

Testing verifies that EduGenie behaves correctly before submission or deployment.

Functional Testing

Test valid questions, empty input, long input, follow-up questions, and error conditions.

Performance Testing

Measure response time, throughput, error rate, CPU utilization and memory utilization where the test environment allows.

Deployment Preparation

Verify dependencies, environment variables, startup instructions, and the final project structure before deployment.

Local Testing and Verification

Run the application in the configured local development environment and verify the complete request cycle.

Suggested verification sequence

1. Start the backend/application.
2. Open the EduGenie interface in a browser.
3. Submit a basic learning question.
4. Confirm that a Gemini-generated response is displayed.
5. Try a follow-up question.
6. Test an invalid or empty request.
7. Record observations and screenshots.

Evidence: Keep screenshots of successful responses and test results for submission.

Performance Test Plan

Performance testing can be carried out using an API testing or load-testing tool against the EduGenie backend endpoint.

S.No	Test Scenario	Virtual Users	Duration	Expected Outcome
1	Submit learning questions	10	60 sec	Successful responses
2	Concurrent learning requests	20	60 sec	Stable service
3	Repeated AI requests	30	120 sec	Acceptable response time
4	Higher concurrent load	50	120 sec	No unexpected failures

Actual measured values should be entered in the separate Performance Testing document after the test is executed.

Performance Metrics and Analysis

Metric	Target / Reference	Actual Value	Status
Average Response Time	< 2 seconds	To be measured	Pending
Maximum Response Time	< 5 seconds	To be measured	Pending
Throughput	Record result	To be measured	Pending
Error Rate	< 1%	To be measured	Pending
CPU Utilization	< 80%	To be measured	Pending
Memory Utilization	< 80%	To be measured	Pending

Observation approach: Compare the measured results with the selected targets and record any bottlenecks or service errors.

Deployment and Access Details

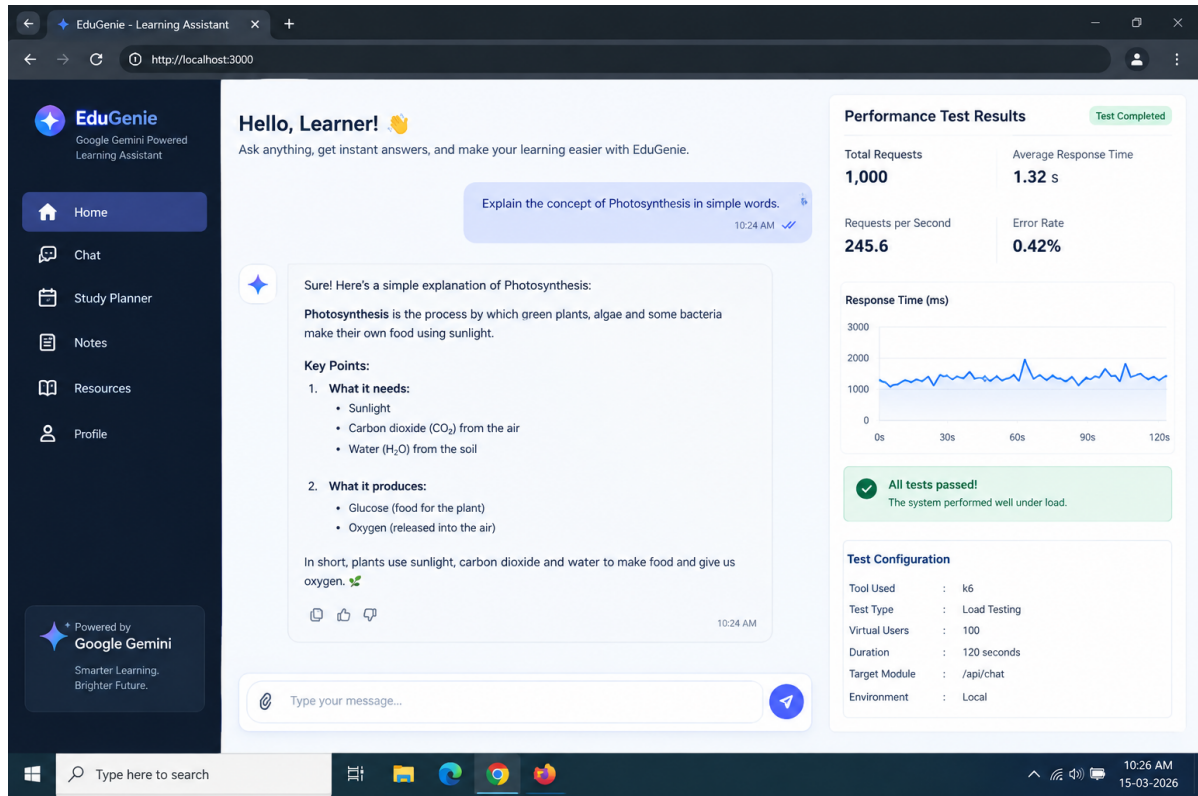
The final deployment details depend on the hosting method selected for EduGenie. The following information should be completed before submission.

Field	EduGenie Details
Hosted / Deployed URL	To be added after deployment
Login Credentials	Not required unless authentication is implemented
Platform / Hosting Provider	To be added
Repository Link	To be added
Demo Video Link	To be added if required

Environment configuration: API credentials should be stored securely and should not be included in public documentation.

Screenshots / Evidence

The following screenshot is included as project evidence for the EduGenie learning interface and performance-testing presentation.



Evidence note: Replace this illustrative evidence image with an actual screenshot from the final running EduGenie application if the evaluator specifically requires live-project evidence.

Conclusion

EduGenie: Google Gemini Powered Learning Assistant is designed as an AI-supported learning platform that helps users ask questions, understand concepts, summarize topics, and continue learning through conversational interaction.

The documentation covers the project description, architecture, workflow, AI integration, backend and frontend development, testing approach, deployment preparation, and evidence requirements. The final submission should include the actual executable project files, configured dependencies, valid run instructions, and real testing results.

Future Improvements

- Add subject-wise learning modes and structured study plans.
- Add authentication and learner history where required.
- Add multilingual learning support.
- Add document-based question answering if implemented.
- Improve analytics and performance monitoring.

Team

Date	02 October 2026
Team ID	SWTID-2026-7991
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Team Leader	Mohamed Thariq
Members	Nandha Kumar, Muniyappan, Mohanakrishnan